

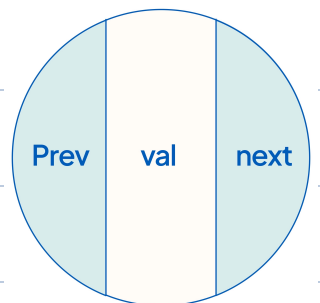
Agenda:

1. Doubly Linked List Introduction
 2. Insert a Node in DLL
 3. Delete the first occurrence of an element in DLL
 4. K Most Recent Integer (LRU Cache)
-

Doubly Linked List Introduction

It is similar to a singly linked list but with an additional feature that each node in a DLL contains references to both the next and the previous nodes in the list.

The previous pointer of the first node always points to null and the next pointer of the last node also points to null in the doubly linked list.



```
class Node {
    int data;
    Node prev, next;

    Node(x) {
        data = x;
        prev = NULL;
        next = NULL;
    }
}
```

Real Life Application - Spotify's Music Manager

Scenerio

Spotify wants to enhance its user experience by allowing users to navigate through their music playlist seamlessly using "**next**" and "**previous**" song functionalities.

Problem

You are tasked to implement this feature using a **doubly linked list** where each node represents a song in the playlist. The system should support the following operations:

- **Add Song**: Insert a new song into the playlist. If the playlist is currently empty, this song becomes the "**Current song**".
- **Play Next Song**: Move to the next song in the playlist and display its details.
- **Play Previous Song**: Move to the previous song in the playlist and display its details.
- **Current Song**: Display the **details** of the current song **being played**.

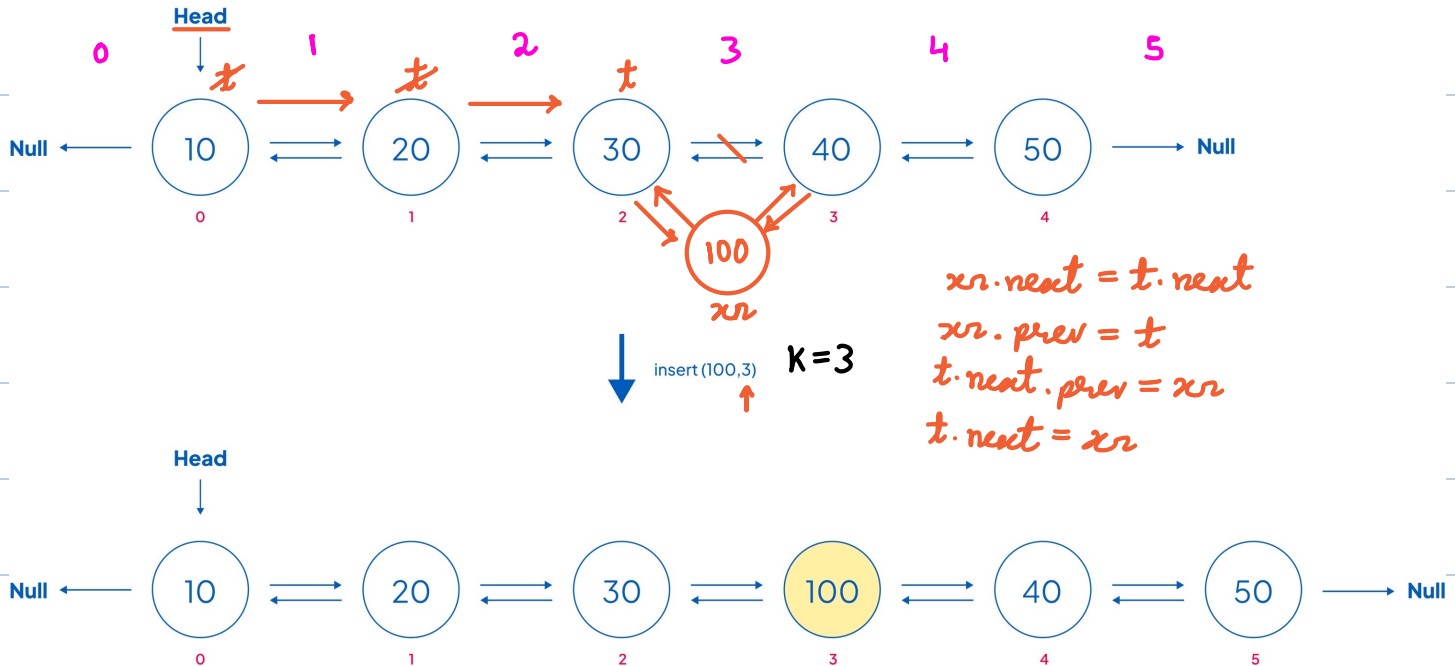
Constraints:

- Each song is uniquely identified by its song ID.
- The playlist starts empty, and the first song added becomes the current song.
- All operations are valid within the current state of the playlist (i.e., there won't be a request to play the next song if there is no next song, and no request to play the previous song if there is no previous song).

Insert a Node in DLL

Q1: Insert an element X at index K in the given doubly linked list.

$$0 \leq K \leq N$$



$xr = \text{new Node}(x)$

if ($K == 0$) {

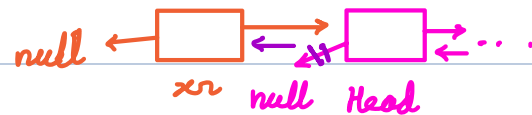
$xr.\text{next} = \text{Head}$

if ($\text{Head} \neq \text{null}$) $\text{Head}.\text{prev} = xr$

$\text{Head} = xr$

return Head

}



$\text{temp} = \text{Head}$

for $i \rightarrow 1$ to $(K-1)$ {

```
    temp = temp.next  
}
```

```
xr.next = temp.next
```

```
xr.prev = temp
```

```
if (temp.next != null) temp.next.prev = xr
```

```
temp.next = xr
```

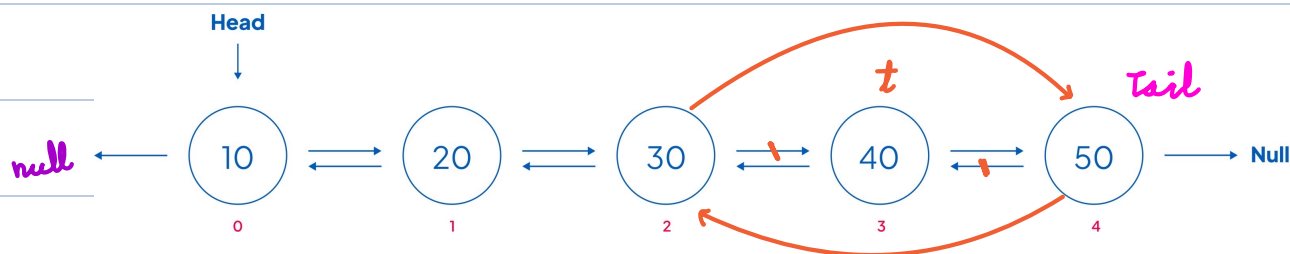
```
return Head
```

TC = $O(K)$

Delete a Node from DLL

Q2: Delete the first occurrence of an element X in the given doubly linked list.

If the element is not present then no change.



Example: X = 40

t.prev.next = t.next

t.next.prev = t.prev

temp = Head

while (temp != null) {

if (temp.data == X) break

temp = temp.next

}

// Search $\rightarrow TC = O(N)$

if (temp == null) return Head

if (temp.next != null) temp.next.prev = temp.prev

if (temp.prev != null) temp.prev.next = temp.next

else Head = temp.next

// Delete $\rightarrow TC = O(1)$

Total TC = $O(N)$

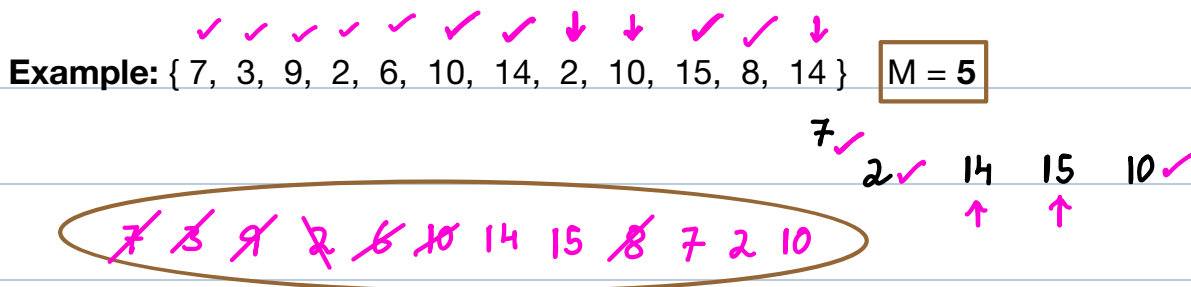
LRU Cache

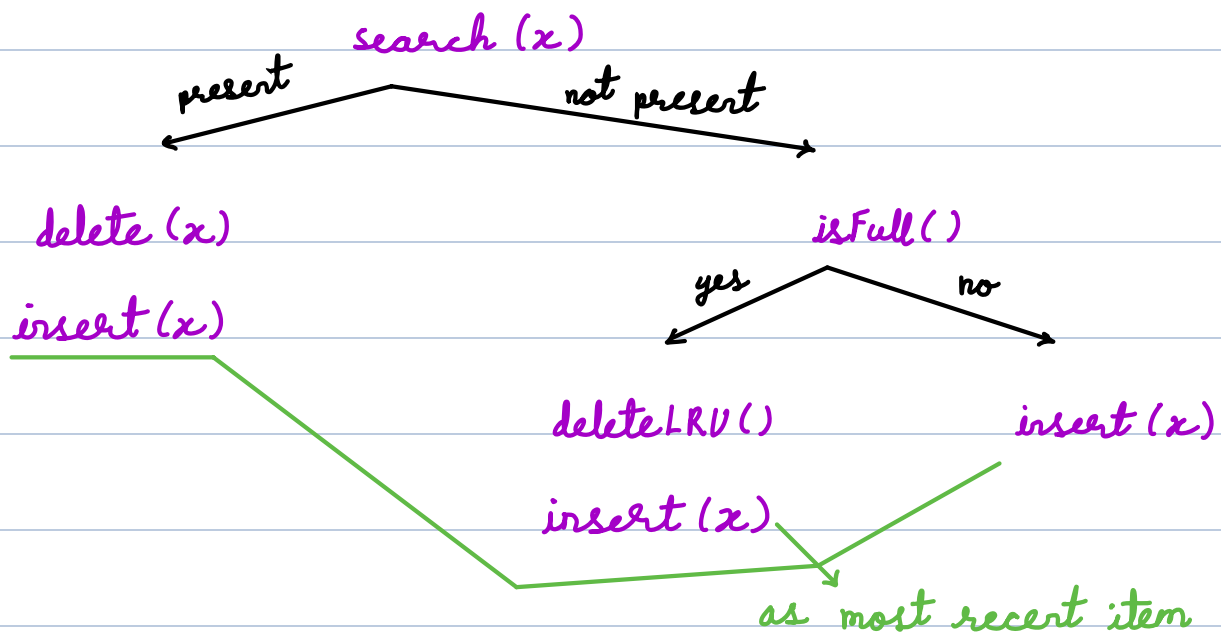
Cache Memory is a special very high-speed memory. The cache is a **smaller and faster memory** that stores copies of the data from **frequently used** main memory locations. There are various different independent caches in a CPU, which store instructions and data. The most important use of cache memory is that it is used to **reduce the average time to access data from the main memory**.

There are different ways of implementing cache, one of them is **LRU cache**.

The **Least Recently Used** (LRU) Cache operates on the principle that the data most recently accessed is likely to be accessed again in the near future. By evicting the least recently accessed items first, **LRU Cache ensures that the most relevant data remains available in the cache**.

Q3: We have been given a **running stream of integers** and the **fixed memory size of $O(M)$** , we have to maintain the **most recent M elements**. In case the current memory is full, we have to delete the least recent element and insert the current data into the memory (as the most recent item).





search(x) → ~~HashSet~~

HashMap

delete TC = $O(N)$

insert as most recent (linear order) → ~~Array~~, ~~Array list~~

delete(x)

~~Linked list~~, Doubly Linked List

~~Stack~~, ~~Queues~~

$x \rightarrow$ node with data x

Example → M = 4

✓ 7 ✓ 2 ✓ 7 ✓ 3 ✓ 5 ✓ 8 ✓ 5 ✓ 3 ✓ 2 ✓ 9

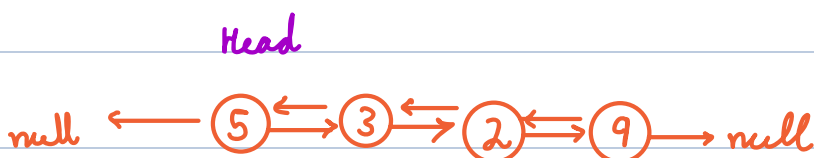
Hash Map

2 → (2)

9 → (9)

3 → (3)

5 → (5)



1) `search(x)` → `hm.containsKey(x)` ✓

2) `insert(x)` [Most Recent] Tail → $\begin{cases} \text{Tail.next} = x \\ \text{Tail} = x \end{cases}$ $x.\text{prev} = \text{Tail}$

3) `delete(x)` → `temp = hm.get(x)` ✓ delete temp from DLL ✓

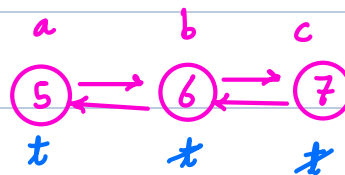
4) `deleteLRU()` [Least Recent] Head → delete Head & remove from HashMap ✓

5) `isFull()` → $\begin{cases} \text{if } (\text{hm.size()} < M) \Rightarrow \text{not full} \\ \text{else full} \end{cases}$

✓ functions → TC = $O(1)$

Predict output of following code:

```
public class Test {  
    public static void main(String[] args) {  
  
        Node a = new Node(5);  
        Node b = new Node(6);  
        Node c = new Node(7);  
  
        a.next = b;  
        b.prev = a;  
  
        b.next = c;  
        c.prev = b;  
  
        Node temp = c;  
  
        while(temp != null) {  
            System.out.print(temp.val + " ");  
            temp = temp.prev;  
        }  
    }  
}
```



7 6 5

An LRU (Least Recently Used) Cache has a capacity of 3 elements.

When the cache exceeds its capacity, it removes the least recently used element. The following operations are performed sequentially:

```
put(1)
put(2)
put(3)
get(1) ✓
put(4) ✓
```

Assume:

- **put(x)** inserts element x into the cache.
- **get(x)** accesses the element and marks it as most recently used.

Which element will be removed from the cache after the last operation?

~~1~~ ~~2~~ 3 1 4